Cover Page

SMART HUB

University Management System

Pre-Defense Project Document

Project Status: Completed Prototype

Prepared for: Pre-Defense Evaluation

Academic Year: 2026

Prepared by: Md Shoeb Abedin

Department: Computer Science & Engineering

Abstract

Abstract

Smart Hub is a web-based University Management System designed to centralize important academic services in a single platform. The system addresses common difficulties students and academic staff face when information is distributed across different communication channels and manual processes. The prototype provides authentication, user and profile management, department and batch management, notices, learning resources, assignments, routines, results, and chat-related infrastructure. The application follows a client-server architecture using a modern web frontend, RESTful backend services, and a relational database. Security is supported through authentication, authorization, password hashing, and token-based access control. The current prototype demonstrates the core academic management workflow and provides a foundation for future extensions such as an AI-powered academic assistant, notifications, attendance, analytics, and mobile applications.

Keywords

University Management System, Smart Hub, Next.js, React, Node.js, Express.js, MySQL, REST API, JWT, Academic Management.

Introduction

1.1 Background

Digital transformation has become increasingly important in higher education. Universities generate and exchange a large volume of information including notices, course materials, assignments, routines, results, student profiles, and departmental information. When these services are handled through disconnected systems, students may spend unnecessary time searching for information and academic staff may face difficulties maintaining consistent records.

1.2 Project Overview

Smart Hub was developed as a centralized platform for university-related academic management. The system brings several common academic operations together and provides role-aware access to users. The prototype is intended to demonstrate how a university can organize academic information through a web application while maintaining a clear separation between frontend, backend, database, and authentication responsibilities.

1.3 Purpose

The purpose of this project is to provide a practical digital solution for managing academic information and to demonstrate the application of modern full-stack development techniques in a university-oriented software project.

Problem Statement

2.1 Existing Problems

Students often receive notices through social media groups, messaging applications, printed notices, or separate portals. Course resources may be stored in different locations. Assignment information may not be centralized, and routine and result information can require repeated checking. Administrators also need reliable mechanisms for maintaining users, departments, batches, and academic content.

2.2 Core Problem

The core problem is the absence of a single, organized platform that connects these academic services while controlling access according to user roles. A fragmented information environment can cause missed notices, duplicate communication, inconsistent records, and unnecessary administrative effort.

2.3 Need for the System

A centralized system can reduce information fragmentation, improve accessibility, and provide a structured foundation for future digital services. Smart Hub addresses this need through a modular web application and REST API architecture.

Objectives

3.1 General Objective

To design and develop a centralized web-based University Management System that simplifies academic information management and provides students, faculty, and administrators with organized access to relevant services.

3.2 Specific Objectives

• Implement secure user authentication and authorization.
• Provide profile and user management.
• Manage departments and batches.
• Publish and view university notices.
• Upload and access learning resources.
• Manage assignments.
• Publish and view class routines.
• Publish and view academic results.
• Provide a scalable REST API backend.
• Store application data in a relational database.
• Establish a foundation for future intelligent academic assistance.

3.3 Expected Outcome

The expected outcome is a functional prototype demonstrating the major workflows of a centralized university management platform with a clear path for future expansion.

Scope of the Project

4.1 In Scope

The current prototype covers authentication, user profiles, departments, batches, notices, resources, assignments, routines, results, and related backend APIs. The system is designed with role-based access in mind so that different users can receive appropriate permissions.

4.2 Out of Scope for Current Prototype

The following are not treated as completed features in the current prototype: production-grade AI assistant, automated attendance hardware integration, full payment processing, native mobile applications, advanced analytics, and large-scale institutional deployment.

4.3 Future Scope

Future versions can introduce AI-based document question answering, notifications, attendance management, online assignment submission, analytics dashboards, mobile applications, and integrations with institutional services.

Existing System Analysis

5.1 Conventional Approach

In many academic environments, information is distributed through multiple channels. Notices may appear on notice boards or social groups, learning resources may be shared through cloud links, and academic records may be managed independently. This approach can work for small amounts of information but becomes difficult to maintain as the number of users and records increases.

5.2 Limitations

Fragmentation, inconsistent updates, limited searchability, repeated manual communication, and difficulty tracking academic information are major limitations. Students may also have difficulty determining which source contains the latest information.

5.3 Proposed Improvement

Smart Hub consolidates major academic information into one application. The centralized design does not eliminate institutional procedures, but it provides a structured digital layer through which academic information can be created, stored, accessed, and managed.

Proposed System

6.1 System Concept

Smart Hub uses a client-server model. The frontend provides the user interface, the backend exposes RESTful endpoints and handles business logic, and the database stores persistent information. Authentication middleware protects routes that require a signed-in user.

6.2 User Roles

Administrator users can manage core system data. Faculty-oriented functionality can support academic content management. Students can access information relevant to their academic activities. The exact permission model can be extended as the prototype evolves.

6.3 Main Workflow

A user authenticates through the frontend. The frontend communicates with the backend through HTTP requests. The backend validates authentication, processes the request, performs required database operations, and returns structured JSON responses. The frontend then renders the result to the user.

Functional Requirements

7.1 Authentication

The system shall allow users to register and log in using valid credentials. Passwords shall not be stored as plain text. Protected endpoints shall require valid authentication information.

7.2 User and Profile Management

Authorized users shall be able to access and update appropriate profile information. Administrative functions may manage user records according to permissions.

7.3 Academic Information

The system shall support notices, resources, assignments, routines, and results. Records shall be stored in the database and retrieved through backend APIs.

7.4 Department and Batch Management

Authorized users shall be able to maintain department and batch information so that academic records can be organized structurally.

7.5 API Services

The backend shall provide modular REST endpoints for the major system modules and return consistent JSON responses for frontend consumption.

Non-Functional Requirements

8.1 Security

The system should protect sensitive endpoints through authentication and authorization, hash passwords securely, validate input, and avoid exposing unnecessary database details to clients.

8.2 Performance

Database queries should be structured to return required data efficiently. The frontend should avoid unnecessary requests and provide responsive feedback during loading operations.

8.3 Usability

The interface should present common academic tasks clearly and reduce unnecessary navigation. Forms should provide understandable validation feedback.

8.4 Maintainability

The backend is organized into routes, controllers, middleware, and database modules. This separation supports easier debugging and future feature development.

8.5 Scalability

The modular architecture provides a foundation for adding additional academic modules without rewriting the entire application.

System Architecture

9.1 Architecture Overview

The prototype follows a three-layer style: presentation layer, application/API layer, and data layer. The browser-based frontend communicates with the Express.js backend using HTTP/REST APIs. The backend communicates with MySQL/MariaDB for persistent storage.

9.2 Architecture Flow

User → Next.js/React Frontend → REST API → Express.js Controllers → Authentication/Business Logic → MySQL/MariaDB → JSON Response → Frontend.

9.3 Backend Organization

The backend uses route modules for authentication, notices, resources, chat-related endpoints, results, departments, profiles, routines, assignments, users, and batches. Controllers contain request-processing logic while middleware provides cross-cutting concerns such as authentication.

9.4 Static and File Handling

The backend can expose uploaded resource files through a static uploads path. File-upload functionality can be implemented with middleware such as Multer.

Technology Stack

10.1 Frontend

Next.js and React are used to build the client-side application. Tailwind CSS is used for responsive interface styling and reusable utility-based layouts.

10.2 Backend

Node.js provides the runtime environment and Express.js provides the HTTP server and REST API framework.

10.3 Database

MySQL/MariaDB is used as the relational database for users, departments, batches, notices, resources, assignments, routines, results, and related records.

10.4 Authentication and Security Libraries

JWT is used for token-based authentication, bcrypt is used for password hashing, dotenv manages environment variables, and CORS controls cross-origin access.

10.5 Development Approach

The project uses modular JavaScript-based development with separate route and controller responsibilities, enabling incremental development and debugging.

Major Modules

11.1 Authentication Module

Handles registration, login, authentication tokens, and protected access.

11.2 Notice Module

Provides creation, retrieval, updating, and management of academic notices.

11.3 Resource Module

Supports academic resource upload and access, including document-based materials.

11.4 Assignment Module

Provides assignment-related academic information and management.

11.5 Routine Module

Provides class routine information to users.

11.6 Result Module

Provides academic result information.

11.7 Department and Batch Modules

Organize academic structure by department and batch.

11.8 Profile and User Modules

Manage user information and profiles.

11.9 Chat Infrastructure

The prototype contains chat-related backend routes and session/message storage infrastructure. The AI assistant is not treated as a completed production feature in this document.

Database Design

12.1 Database Approach

A relational database is appropriate because university data contains structured relationships among users, departments, batches, academic resources, assignments, routines, and results.

12.2 Representative Entities

Representative entities include users, departments, batches, notices, resources, assignments, routines, results, chat_sessions, and chat_messages. Exact table names and fields may vary with the final implementation.

12.3 Relationships

A user may belong to an academic department and batch. Resources, assignments, routines, and results can be associated with academic structures or users depending on the business rules. Chat sessions belong to users and chat messages belong to sessions.

12.4 Data Integrity

Foreign keys and validation rules should be used to prevent invalid references. Application-level checks can provide additional protection before inserting related records.

Security & Authentication

13.1 JWT Authentication

After successful authentication, the backend can issue a JSON Web Token containing the required identity information. Protected routes use authentication middleware to verify the token before allowing access.

13.2 Password Security

Passwords are hashed using bcrypt before storage. The original password should never be stored or returned in API responses.

13.3 Authorization

Authentication confirms identity, while authorization determines whether the authenticated user is permitted to perform an operation. Role-aware authorization is important for administrative operations.

13.4 CORS

The backend includes an explicit CORS configuration to allow trusted frontend origins while rejecting unauthorized browser origins.

13.5 Environment Variables

Database credentials, JWT secrets, AI service configuration, and other deployment-specific values should be stored in environment variables rather than committed directly to source code.

Implementation & API Design

14.1 REST API

The backend follows REST-style endpoints grouped by module. Examples include /api/auth, /api/notices, /api/resources, /api/results, /api/departments, /api/profile, /api/routine, /api/assignments, /api/users, and /api/batches.

14.2 Request Flow

A request enters Express, passes through CORS and body parsing, reaches the relevant route, and then passes through authentication middleware where required. The controller performs validation and database operations before returning a response.

14.3 Error Handling

The prototype includes centralized error handling so that unexpected errors can be logged and returned as structured responses. Production deployment should avoid exposing sensitive internal error information.

14.4 File Uploads

Academic resources can be uploaded and served through a controlled uploads directory. File type, size, naming, and access rules should be validated for production use.

Testing & Evaluation

15.1 Testing Approach

Testing focuses on authentication, protected routes, CRUD operations, database interactions, API responses, file handling, and frontend-backend communication.

15.2 Functional Test Cases

1. Register with valid data — expected: account created.
2. Login with valid credentials — expected: authenticated response.
3. Login with invalid credentials — expected: rejection.
4. Access protected endpoint without token — expected: unauthorized response.
5. Create and retrieve notice — expected: record appears correctly.
6. Upload resource — expected: resource is stored and accessible.
7. Retrieve routine/result/assignment — expected: relevant records are returned.
8. Access another user's protected data — expected: permission rules prevent unauthorized access where applicable.

15.3 Current Evaluation

The project is currently a completed prototype. Core modules have been implemented for demonstration and academic evaluation. Further testing with larger datasets and multiple concurrent users would be required before production deployment.

Limitations & Future Work

16.1 Current Limitations

The current prototype is not positioned as a full enterprise university platform. Advanced reporting, institutional integrations, comprehensive attendance, production-grade notifications, and large-scale deployment optimization remain future work.

16.2 AI Assistant Future Work

An AI-powered academic assistant is a planned enhancement. A document-grounded approach such as Retrieval-Augmented Generation can be explored so that the assistant answers questions using approved university resources rather than relying only on general model knowledge.

16.3 Additional Future Features

Future releases may include attendance, online assignment submission, email and push notifications, advanced analytics, mobile applications, teacher dashboards, automated reminders, and improved administrative reporting.

Conclusion

17.1 Conclusion

Smart Hub demonstrates the feasibility of a centralized University Management System using modern full-stack web technologies. The completed prototype brings together authentication, user management, academic content, resources, assignments, routines, results, and institutional structures in a single platform.

17.2 Contribution

The major contribution of the project is the integration of several academic workflows into one modular web application. The architecture also provides a practical foundation for future features without requiring a complete redesign.

17.3 Final Statement

The project meets the primary prototype objectives and provides a strong basis for future development. With additional security hardening, performance testing, institutional requirements, and intelligent academic assistance, Smart Hub can be extended toward a more comprehensive university digital platform.

References & Pre-Defense Notes

18.1 Technical References

• Next.js Documentation — Official framework documentation.
• React Documentation — Official UI library documentation.
• Node.js Documentation — Official runtime documentation.
• Express.js Documentation — Official server framework documentation.
• MySQL Documentation — Official relational database documentation.
• MariaDB Documentation — Official database documentation.
• JSON Web Token (JWT) Documentation — Token-based authentication concepts.
• bcrypt Documentation — Password hashing concepts.

18.2 Presentation Notes

For the pre-defense presentation, emphasize the problem, proposed solution, architecture, major modules, database design, technology choices, completed prototype, testing approach, limitations, and future work. The AI assistant should be presented as future work unless the implementation is fully completed and verified.

18.3 Demo Flow

Recommended demonstration order: Login → Dashboard → Profile → Notices → Resources → Assignments → Routine → Results → Department/Batch management → Explain architecture → Explain database → Discuss security → Explain limitations and future work.

18.4 Final Pre-Defense Checklist

• Verify frontend and backend run correctly.
• Verify database connection.
• Verify login/logout.
• Verify major CRUD modules.
• Prepare screenshots of important pages.
• Prepare architecture and ER diagrams.
• Prepare a short live demo.
• Keep backup screenshots/video in case of deployment issues.
• Review likely viva questions on technologies and design decisions.